The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2], and a computationally demanding one. The most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges, and on the graphs to which they are now routinely applied that quantity is large [7].

Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original. The strongest guarantees are spectral. Spielman and Srivastava [8] show that sampling each edge with probability proportional to its effective resistance, and reweighting the retained edges by the inverse of that probability, yields a subgraph whose Laplacian quadratic form approximates that of the original within a factor $1 \pm \epsilon$ [9], [10]. Spectral approximation is a statement about the Laplacian, and the eigenvectors of the Laplacian encode cuts and connectivity [11], [12]. Community structure is closely related to those quantities without being identical to them. Because effective resistances are themselves expensive to compute, practical variants approximate them. DSpar [13] replaces the resistance of an edge (u, v) by $1/d_u + 1/d_v$, which bounds it within a factor $2/\alpha$ where α is the normalized spectral gap [14], and thereby obtains a spectral sparsifier at the cost of reading the degree sequence. If the properties preserved by sparsification include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. Whether they do is the question this paper addresses. The answer we obtain is conditional. On the networks and detection algorithms examined here, two conditions have to hold together: the algorithm must take the number of communities as an input, and the graph must carry edges whose removal does not destroy its community structure. Where either condition fails, we find no benefit of any kind, in quality or in speed.

Satuluri et al. [15] introduced local similarity sparsification, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. They reported speedups commonly between 10× and 50× with little or no loss of cluster quality, and improved accuracy for two of the four clustering algorithms they tested.

The approach was taken up and extended. Hamann et al. [16] conducted a systematic comparison of structure-preserving sparsification methods for social networks and released reference implementations, which are now distributed in standard graph libraries. Wu and Chen [17] trained a generative model to sparsify graphs specifically for community detection, and report that clustering accuracy is retained while as little as five percent of the edges are kept. Sparsification also entered production: the SimClusters system at Twitter [18] prunes its interaction graph before community detection at industrial scale. Most recently, Chen et al. [19] benchmarked twelve sparsification algorithms against sixteen graph properties on fourteen networks, concluding that no single sparsifier preserves all properties and that the choice should be matched to the downstream task.

The conditions under which the original result was obtained are specific, and none of them travelled with it. The accuracy gain was reported for fixed-k cut partitioners, judged against external ground-truth communities, on dense graphs, and against clustering runs that took hours. The pipeline is now applied to modularity optimizers that choose their own granularity, judged partly by objectives computed on the graph itself, on sparse graphs, and against detection algorithms that run in near-linear time. The same paper reports a synthetic sweep in which the method begins to outperform clustering on the unsparsified graph at an average degree of approximately 50, and to our knowledge no subsequent work examines that threshold.

These extensions also change what has to be measured. The question a sparsify-then-detect pipeline is meant to answer is whether the communities of the original graph can still be recovered after most of its edges are removed. The object of interest is therefore the original graph, and the quantity to be compared is the quality, on that graph, of two partitions: the one obtained by running the detection algorithm on the sparsified graph, and the one obtained by running it on the original graph directly. Both partitions cover the same vertex set, so both can be scored on the same object, and the difference between those two scores is the effect of sparsification.

The sparsifier classes and detection algorithms described above each impose further requirements of the same kind,

The authors are with the Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA. E-mail: {md724, bd344, ikoutis, bader}@njit.edu

and several have been noted individually in the literature. Similarity-based sparsifiers [15] remove between-community edges preferentially, and those are the edges that modularity and conductance penalize, so the sparsifier and the quality measure act on the same quantity. Degree-based sparsifiers [13] select on endpoint degrees, and a heavy-tailed degree sequence reproduces several of the resulting effects with no community structure present, so a degree-preserving random graph [20] is needed to separate the two. Detection algorithms that choose their own granularity [4], [5] return finer partitions on a sparsified graph: Chen et al. [19] document community counts rising by three orders of magnitude across a pruning sweep, and Hamann et al. [16] warn that normalized mutual information is inflated when sparsification fragments the graph, recommending chance-corrected indices in its place. Finally, sparsification is itself a computation, and Gottesbüren et al. [21] observe that its overhead can counteract the speedup it is intended to produce. Each requirement is a property of a specific method class rather than a general caveat, and each bounds what a comparison involving that class can conclude. Section III states the requirements and the control that establishes each one.

Applying these requirements, we re-examined the question across seven sparsifiers, spanning the degree-based and similarity-based families that have claimed quality gains, the method of Satuluri et al. among them, and a connectivity-preserving class that cannot fragment a graph by construction; four detection algorithms drawn from the two families identified above; seventeen real networks of up to 25 million edges; and a synthetic benchmark in which average degree is swept over an order of magnitude while community structure is held fixed. One arm of the synthetic study uses a fixed-k partitioner, so that the number of communities is identical in the sparsified and unsparsified conditions and granularity is excluded by construction rather than by matching. The study comprises 22 controlled experiments. Predictions and stopping rules were fixed before each experiment was run, and several of our own hypotheses did not survive them.

Of the two conditions, the second has a location. Average degree is the correlate we can vary under control, and measured against it the transition between the two regimes is sharp within a given family of graphs while its location is not universal. On the LFR benchmark graphs [22] we generate it falls near an average degree of 50, which is also the threshold reported in the original synthetic sweep. On the real networks we test it has already occurred at an average degree of 29, the lowest density at which we tested a fixed-k partitioner. We therefore state a condition rather than a threshold. Density itself is unlikely to be the operative variable. Injecting spurious edges into a graph makes sparsification more effective, which points instead to how many edges can be removed without destroying community structure. Degree heterogeneity and the noise present in how a graph was collected are further correlates of that quantity, and we do not separate them here.

Where either condition fails, no benefit of any kind is present. For detection algorithms that choose their own granularity, on the sparse real networks we test, no sparsifier improves modularity measured on the original graph beyond run-to-run variation, at any retention that preserves quality, and this holds for every such algorithm we tested. There is no speed benefit either. Measured end to end, including the cost of the sparsifier, the pipeline is no faster than clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget, with one exception on one network, where the surviving recovery gain is instead exceeded by an unsparsified run of a different detection algorithm.

Where both conditions hold, the gain is real. On synthetic graphs with planted communities, with the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels. Recovery turns positive at a lower density than the objective does. Degree-based and uniform sparsification at identical retention do not reproduce the effect, so it belongs to the similarity signal rather than to the edge budget.

A fixed-k partitioner applied to a sparse graph shows no gain on the objective in any configuration we tested, and a free-granularity optimizer applied to a dense one shows none either once granularity is controlled. Neither condition is sufficient on its own. Which sparsifier is used matters less than either condition, provided its selection signal reflects local structure rather than degree alone. The mechanism is consistent with this: what governs a degree-based sparsifier's effect on modularity is where high-degree-product edges sit relative to community boundaries, which we establish by direct intervention.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one. On the synthetic benchmarks, a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured, and the same holds on all but one of the real networks we test. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. The conditions under which sparsification helps, namely a detection algorithm that takes the number of communities as an input and a graph carrying sufficient removable redundancy, established under controls that exclude granularity by construction, together with the negative territory delimited across seven sparsifiers and four detection algorithms on seven real networks. The negative result is not an artifact of the fragmentation that

sparsification induces: a connectivity-preserving sparsifier that cannot disconnect a graph leaves no vertex in a small component, and modularity falls by the same margin. We also report that these methods have a hard retention floor on sparse graphs, so that aggressive sparsification is not merely unhelpful there but unreachable.

- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and modularity are all held exactly fixed.
- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

References

[1] S. Fortunato, "Community detection in graphs," Physics Reports, vol. 486, no. 3–5, pp. 75–174, 2010.

[2] S. Fortunato and M. E. Newman, "20 years of network community detection," Nature Physics, vol. 18, no. 8, pp. 848–850, 2022.

[3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical Review E, vol. 69, no. 2, p. 026113, 2004.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008.

[5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, pp. 1–12, 2019.

[6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences, vol. 105, no. 4, pp. 1118–1123, 2008.

[7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," Internet Mathematics, vol. 6, no. 1, pp. 29–123, 2009.

[8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, 2008, pp. 563–568.

[9] A. A. Benczúr and D. R. Karger, "Randomized approximation schemes for cuts and flows in capacitated graphs," SIAM Journal on Computing, vol. 44, no. 2, pp. 290–319, 2015.

[10] D. A. Spielman and S.-H. Teng, "Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems," SIAM Journal on Matrix Analysis and Applications, vol. 35, no. 3, pp. 835–885, 2014.

[11] U. von Luxburg, "A tutorial on spectral clustering," Statistics and Computing, vol. 17, no. 4, pp. 395–416, 2007.

[12] J. Shi and J. Malik, "Normalized cuts and image segmentation," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, pp. 888–905, 2000.

[13] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, "Dspar: An embarrassingly simple strategy for efficient GNN training and inference via degree-based sparsification," Transactions on Machine Learning Research, 2023.

[14] L. Lovász, "Random walks on graphs," Combinatorics, Paul Erdős Is Eighty, vol. 2, no. 1–46, p. 4, 1993.

[15] V. Satuluri, S. Parthasarathy, and Y. Ruan, "Local graph sparsification for scalable clustering," in Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 721–732.

[16] M. Hamann, G. Lindner, H. Meyerhenke, C. L. Staudt, and D. Wagner, "Structure-preserving sparsification methods for social networks," Social Network Analysis and Mining, vol. 6, no. 1, p. 22, 2016.

[17] H.-Y. Wu and Y.-L. Chen, "Graph sparsification with generative adversarial network," in 2020 IEEE International Conference on Data Mining (ICDM), 2020.

[18] V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, and J. Lin, "SimClusters: Community-based representations for heterogeneous recommendations at Twitter," in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2020, pp. 3183–3193.

[19] Y. Chen, H. Ye, S. Vedula, A. Bronstein, R. Dreslinski, T. Mudge, and N. Talati, "Demystifying graph sparsification algorithms in graph properties preservation," Proceedings of the VLDB Endowment, vol. 17, no. 3, pp. 427–440, 2024.

[20] M. Molloy and B. Reed, "A critical point for random graphs with a given degree sequence," Random Structures & Algorithms, vol. 6, no. 2–3, pp. 161–180, 1995.

[21] L. Gottesbüren, N. Maas, D. Rosch, P. Sanders, and D. Seemaier, "Linear-time multilevel graph partitioning via edge sparsification," in 33rd Annual European Symposium on Algorithms (ESA), 2025, arXiv:2504.17615.

[22] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," Physical Review E, vol. 78, no. 4, p. 046110, 2008.